

Token-Based Clone Detection for PHP: An Empirical Evaluation of Methods and a Labelled Benchmark

Luciano Federico Pereira

ORCID: 0009-0002-4591-6568

June 2026

Abstract

`phpcpd` is a widely used PHP copy/paste detector whose last released version (6.0.3, 2020) performs Rabin–Karp exact matching of Type-1 and Type-2 clones. A subsequent, unreleased 7.0.0 development line added a ConQAT-style approximate suffix-tree algorithm for Type-3 (near-miss) detection (contributed upstream as pull request #199); however, 7.0.0 was never released and the project was archived in 2023, so this capability did not reach users. The clone-detection literature offers further methods — order-invariant token-bag matching and type-aware normalization — that had not been applied to PHP, and, unlike Java (BigCloneBench) or C (POJ-104), no labelled PHP clone benchmark existed on which to evaluate any of them. This paper reports an empirical study based on the archived `phpcpd` codebase. We (i) modernize it reproducibly to PHP 8.5, PHPStan level 10, and a 159-test suite, and complete and evaluate its previously unreleased suffix-tree engine; (ii) implement additional literature-grounded methods as selectable engines — a SourcererCC token-bag matcher, type-aware edit weights, a token-type abstraction with a `--type-anchored` variant, an IDF stopwords filter, and an incremental index that provably reproduces a full pass; and (iii) construct **BCB-PHP**, a labelled PHP clone benchmark (to our knowledge the first), comprising a mutation-injection harness over a pinned, six-corpus type-density gradient. Evaluating each method on BCB-PHP yields a type-density $\times$ clone-density map, a specificity increase of +1.9 to +18.9 points from type-anchored normalization at no recall cost, and confirmed inconsistent clones in an application corpus. The results motivate a combined exact-and-reorder-tolerant default configuration (**rk+tb**); identifier fuzzing is shown to reduce precision substantially, and approximate suffix-tree matching to be better suited to targeted analysis than to default use. BCB-PHP is released for reuse; its type-density gradient supports further study of clone detection on modern typed versus traditional untyped PHP and of differences between methods.

Note on revision (2026-09-15)

Three of this paper’s measurements described an engine that had since moved, and are corrected in place rather than annotated: the comparison against the original `phpcpd.phar` (§7), which a change to the token model made incommensurable at a shared threshold; a boundary statistic that turned out to measure its own instrument rather than the engine; and the precision section (§6.3), which gains a second study on public corpora rather than losing the one it had. One configuration the paper recommended, **tb+fuzzy**, no longer exists and the recommendation is withdrawn with its justification given below. Every table in this paper is typeset from a committed result file rather than transcribed, and each reproduces from the harness that wrote it. Nothing in the conclusions changed direction.

1 Introduction

`phpcpd` is a carefully engineered PHP copy/paste detector developed by Sebastian Bergmann: a Rabin–Karp rolling hash over a normalized token stream that hashes fixed windows, reports collisions, and detects Type-1 and Type-2 clones. It is fast, dependency-light, and widely used. Its last released version, 6.0.3 (2020), implements this single Rabin–Karp strategy and does not attempt Type-3 (near-miss/gapped) or reorder-tolerant detection — an appropriate scope for an exact-match detector.

A subsequent development line did go further: an unreleased 7.0.0 added a ConQAT-style approximate suffix-tree algorithm with a configurable edit budget for Type-3 detection [2], contributed upstream as pull request #199. That release, however, was never published — its changelog entry carries an unfilled date — and the repository was archived in 2023, so the capability remained in the codebase but absent from any release and unevaluated in practice. The clone-detection literature offers further methods still: SourcererCC’s order-invariant token-bag matching for reordered code [3] and Toma’s token-type abstraction for stronger Type-2 normalization [4], neither of which had been applied to PHP. Evaluating any of these on PHP faced a further obstacle: unlike Java (BigCloneBench [12]) or C (POJ-104), no labelled PHP clone benchmark existed.

Approach. Building on the archived `phpcpd` codebase, we modernize it, complete and evaluate its previously unreleased suffix-tree engine, implement additional literature-grounded methods, and construct the benchmark required to evaluate them. We then determine empirically which methods are appropriate for a default configuration in a fast command-line tool. The evaluation indicates that a combined exact-and-reorder-tolerant configuration is the most suitable default, that identifier fuzzing substantially reduces precision, and that approximate suffix-tree matching is better reserved for targeted analysis.

Contributions.

1. A **diff-level reproducible modernization** of an archived PHP tool to PHP 8.5, PHPStan level 10, PER-CS 2.0, and a 159-test suite, including the completion and first evaluation of `phpcpd`’s previously unreleased ConQAT suffix-tree engine (§4).
2. **Five detection methods implemented** as selectable engines on the inherited base: a SourcererCC token-bag matcher with IDF filtering, type-aware edit weights, an extended token-type abstraction with a type-anchored variant, inconsistent-clone reporting, and an incremental index — each tied to a primary reference and an end-to-end test (§2, §5).
3. **BCB-PHP, a labelled PHP clone benchmark** (to our knowledge the first): a mutation-injection harness over a pinned, six-corpus type-density gradient (§8), released for reuse — its gradient supports further study of modern typed vs. traditional untyped PHP and of method differences.
4. An **empirical evaluation** on BCB-PHP yielding a type-density×clone-density map, a +1.9 to +18.9 point specificity lift from type-anchored normalization at zero recall cost, a six-corpus method comparison, and real inconsistent clones in an application corpus (§8) — from which we recommend the `rk+tb` default (§7).

2 Background: the `phpcpd` Codebase and Its Detection Methods

2.1 The released base engine (Rabin–Karp)

`phpcpd`’s last released version (6.0.3) provides a single strategy, `DefaultStrategy`, a Rabin–Karp matcher. It normalizes tokens (whitespace and comments stripped; `--fuzzy` rewrites `T_VARIABLE` to a placeholder), builds a per-token rolling signature, hashes fixed windows of `--min-tokens` tokens, and reports colliding windows filtered by `--min-lines`. It detects Type-1 and Type-2 exact-match clones and, by construction, tolerates neither gaps nor reordering.

2.2 The inherited suffix-tree engine (unreleased 7.0-dev)

The repository’s unreleased 7.0.0 line, which we fork, introduces a second strategy through a selector in `Application.php` (upstream code; `tokenbag` is the only branch we add):

```
return match ($algorithm) {
    null, 'rabin-karp' => new DefaultStrategy($config),    // selector: upstream 7.0-dev
    'suffixtree'      => new SuffixTreeStrategy($config), // phpcpd base engine
    'tokenbag'        => new TokenBagStrategy($config),   // upstream, PR #199
    'tokenbag'        => new TokenBagStrategy($config),   // added in phpcpd-next
};
```

The `SuffixTreeStrategy` → `ApproximateCloneDetectingSuffixTree` (an implementation of the ConQAT algorithm [2], contributed upstream as PR #199) tokenizes into an `AbstractToken[]` word,

appends a sentinel, builds a generalized suffix tree, and in `findClones($minLength, $maxErrors, $headEquality)` compares candidate token sequences by *approximate* match. A Levenshtein edit-distance matrix `edBuffer` is maintained over token sequences; a candidate extends only while the running edit distance stays $\leq \text{maxErrors}$ (the `--edit-distance` flag, default 5), and `headEquality` enforces an exact-match prefix before edits are allowed, as in ConQAT. Comparing by edit distance rather than hash equality, this method detects Type-3 (near-miss/gapped) clones: inserted, deleted, or substituted statements within the edit budget are tolerated. Because 7.0.0 was never released, this engine had not been evaluated; we modernize it (§4), repair two latent defects it exposed (§5), and evaluate it (§8).

Definition 1 (token edit distance). For token sequences $a_{1..m}, b_{1..n}$, the matcher fills D with

$$D[i, j] = \min \begin{cases} D[i-1, j] + 1 & \text{(deletion)} \\ D[i, j-1] + 1 & \text{(insertion)} \\ D[i-1, j-1] + c(a_i, b_j) & \text{(match/substitution)} \end{cases}$$

and reports a clone while $\min_k D[\cdot, k] \leq \text{maxErrors}$. With uniform $c \equiv 1$ this is plain Levenshtein; §5 (R2) replaces c with a type-aware cost.

Lemma 1 (band sufficiency). $D[i, j] \geq |i - j|$, so $|i - j| > \text{maxErrors}$ implies $D[i, j] > \text{maxErrors}$ and no alignment of cost $\leq \text{maxErrors}$ passes through (i, j) . *Proof.* Aligning an i -token prefix with a j -token prefix requires at least $|i - j|$ insertions or deletions, hence $D[i, j] \geq |i - j|$; the threshold and path claims follow. $\square$ Consequently the cells outside the band $|i - j| \leq \text{maxErrors}$ cannot affect any reported clone, and computing only the band $-2\text{maxErrors} + 1$ cells per diagonal over L steps — costs $O(L \cdot \text{maxErrors})$ rather than $O(L^2)$ with identical output. This justifies the banded optimization of §7.

2.3 Methods added in phpcpd-next

Beyond modernizing and evaluating the inherited engines, we contribute five methods grounded in the literature: a SourcererCC token-bag matcher [3], type-aware edit weights, an extended token-type abstraction [4] with a type-anchored variant, inconsistent-clone reporting, and an incremental index. Each is implemented behind a flag and pinned by an end-to-end test (§5). Two findings from the literature (§3) guide which methods are appropriate by default, and the benchmark of §8 decides it.

2.4 Clone-type coverage

Using the Roy & Cordy taxonomy [1] (Type-1 identical up to layout; Type-2 up to renamed identifiers/literals/types; Type-3 near-miss/gapped; Type-4 semantic), Table 1 contrasts phpcpd’s base engine with the methods phpcpd-next adds. Note on Type-2: phpcpd’s `--fuzzy` abstracts only `T_VARIABLE`; the shared `TokenNormalizer` we add (R3) provides full Type-2 normalization under `--fuzzy` for every method.

Table 1: Clone-type coverage: phpcpd’s base Rabin–Karp engine vs. the methods added in phpcpd-next.

Type	phpcpd base (Rabin–Karp)	phpcpd-next (added methods)
1	✓	✓
2	partial (<code>--fuzzy, T_VARIABLE</code> only)	✓ (full, via <code>TokenNormalizer</code>)
3	— (exact hash)	✓ (unified engine, banded alignment with the divergent ranges named; token-bag for reorder)
4	—	—

3 Two Literature Findings That Guide Method Selection

Finding 1 — simple clones dominate [4]. In real-world OSS most clones are Type-1/2; Type-3 is a meaningful minority; Type-4 is rare. Approaches that sacrifice scalability to chase Type-4 “may not be practical in real life.” For a fast CLI linter, **Type-3 is the ceiling worth reaching; Type-4 is out of scope.**

Finding 2 — the dangerous clones are *inconsistent* ones [2]. Bug risk concentrates where one copy of a near-miss clone was patched and its sibling was not. The value of Type-3 detection is therefore not recall for its own sake but **surfacing diverging copies**.

Joint implication. The methods worth adding are those that make Type-3 detection *better, faster, and usable by default for bug-hunting* — approximate and reorder-tolerant matching with inconsistency reporting — not neural Type-4 machinery. These two findings set the selection criterion the benchmark then applies.

4 Modernization Methodology

The revival is recorded diff-by-diff as a primary source (not retrospectively). The citable methodology:

1. **Platform-first.** `composer.json` pins `platform.php = 8.5` so dependency incompatibilities surface at install time, not runtime.
2. **Bugs before idioms.** Six genuine correctness fixes (an `empty($object)` always-false guard; an unguarded `preg_replace` null return; an `averageSize()` division-by-zero; `current()` on an empty array; ...) landed *before* any cosmetic modernization.
3. **One idiom at a time**, each with a diff and a rationale: typed properties $\rightarrow$ `readonly class` $\rightarrow$ constructor promotion $\rightarrow$ typed constants $\rightarrow$ `#[\Override]` $\rightarrow$ PHPDoc generics.
4. **Static analysis as a ratchet.** PHPStan driven $64 \rightarrow 0$ errors, then the level raised from 9 to `max` (level 10). The gate only tightens.
5. **Dependency re-unification and decoupling.** The four `sebastian/*` runtime packages were replaced with owned code, giving a zero-runtime-dependency tool decoupled from the PHPUnit release train.
6. **Tests last, and they paid off immediately.** The suite surfaced a latent bug invisible to static analysis: `SuffixTreeStrategy::postProcess()` threw on an empty file list, a valid call path.

Measured methodological claim: the modernization reached PHPStan level 10 with zero errors and *still* harboured a latent runtime bug that only a test exercised — a concrete data point that static analysis and testing are complementary, not redundant.

5 Methods Implemented and Evaluated

With the inherited suffix tree reaching Type-3 (§2), we examine its limitations and add further methods. Each is implemented behind a flag and pinned by an end-to-end test; Table 2 summarizes. The set is bounded by a marginal-returns criterion: a method is admitted while the capability it adds outweighs its marginal cost in precision or runtime, and the three methods for which that balance reverses (R6–R8) are declined rather than deferred.

A limitation of the suffix tree: intra-block reordering. The suffix tree models a clone as one approximately-matching *contiguous* substring of a linearised token stream. A swapped pair of statements looks like a delete followed by an insert — two operations that consume at least 2 from `maxErrors`; at the default budget of 5 a few reorderings exhaust it. A suffix tree on a linear token stream *cannot* be order-invariant within a block. A token-multiset (bag-of-tokens) model is order-invariant by construction, which is why it complements the suffix tree rather than duplicating it.

Definition 1 (token-bag overlap clone (SourcererCC [3])). Represent each block as a token multiset. Blocks A, B are clones iff their multiset intersection satisfies

$$|A \cap B| \geq \theta \cdot \max(|A|, |B|), \quad \theta = 0.7 \text{ by default,}$$

computed via an inverted index (token $\rightarrow$ blocks). This is order-invariant: if B is A with two statements swapped, $A \cap B = A$ and the score is 1.

Definition 2 (IDF stopword filter). Let N be the number of blocks and $\text{df}(t)$ the number of blocks containing token t . A token is a *stopword* (excluded from the index and from $|A \cap B|$) iff

$$\frac{\text{df}(t)}{N} > 0.6, \quad N \geq 5.$$

Under `--fuzzy`, normalized placeholders (`T_VARIABLE:$`, `T_STRING:ID`) occur in almost every PHP body and carry no discriminative power; removing them prevents false-positive inflation. The filter is disabled below five blocks so micro-fixtures are unaffected.

Each method below is **implemented and tested**; Table 2 summarizes.

R1 — token-bag + inverted index + IDF (SourcererCC). `TokenBagStrategy` (selectable via `--algorithm tokenbag`). Blocks are function/method bodies (brace-tracked, no parser); clones are overlap pairs gathered through the inverted index. *Complementarity verified end-to-end*: a function with two statements swapped is detected by the token bag while both Rabin–Karp and the suffix tree (`--edit-distance 5`) miss it at min-tokens 25. *Precision*: the IDF filter reduced a self-application scan of the tool’s own `src/` from 17 reported pairs (all false positives) to 0.

R2 — type-aware edit weights. `SubstitutionCost` replaces the uniform $c \equiv 1$ in D with $c = 0$ (equal), $c = 1$ (cosmetic identifier/literal change), $c = 2$ (control-keyword change). Verified: at min-tokens 35 a cosmetic change (`$scaled` $\rightarrow$ `$banner`) is bridged at `--edit-distance 1` while a structural change (`if` $\rightarrow$ `while`) needs 2.

R3 — token-type abstraction (Toma [4]). `TokenNormalizer` maps tokens to ~ 15 classes upstream of *both* engines under `--fuzzy`. This gave the suffix tree the Type-2 detection it never had.

R4 — inconsistent-clone reporting. `isGapped()` (set by `occurrencesDiffer()`) distinguishes exact from gapped clones; `numberOfGappedClones()` aggregates them; the text logger marks each gapped clone [`inconsistent`]. As implemented here it used data the suffix-tree matcher already computed and previously discarded; since that engine’s removal in 2.0.0 the same flag is set by the unified engine’s banded alignment, which additionally names the token range that diverged on each side rather than only that one did.

R5 — incremental / index-based detection (Hummel et al. [5]). `IncrementalIndex` persists each file’s tokenization keyed by a content hash and re-tokenizes only changed files, replaying detection from the index for the rest (`--incremental`). The key property:

Proposition 1 (incremental equivalence). *Let idx map each file f to its tokenization $T(f)$ keyed by content hash $h(f)$, and let $F' \subseteq F$ be the files whose hash changed since the last run. If detection is replayed from $\{T(f) : f \in F\}$ — recomputing $T(f)$ only for $f \in F'$ and reading the rest from idx — then for every file set F and configuration c the result is bit-identical to a full Rabin–Karp pass over F . Proof sketch. T is a pure function of file content and c , so a cache hit (h unchanged) returns exactly the $T(f)$ a full pass would compute; detection consumes only the $T(f)$ and merges them deterministically, so identical inputs yield identical output. $\square$ Only the *work* is incremental, never the *answer*: per-commit cost moves from $O(\text{corpus})$ toward $O(\text{diff})$. Pinned as the headline test of `IncrementalIndexTest`.*

Duplicated-line accounting. A duplicated-line percentage is a coverage measure, so it is computed as one: the number of source lines belonging to at least one clone, over the number of source lines scanned. This is the clone coverage of ConQAT and Teamscale [2], and it is bounded by the size of the code by construction.

Earlier revisions of this work summed a per-clone contribution instead, adding ℓ lines for each copy of a clone of ℓ lines beyond the first. That is not a coverage measure and does not behave like one. It double-counts a line covered by two overlapping clones, so a single file could be reported as 440% duplicated; it undercounts where a longer clone begins on a line a shorter one has already claimed, reporting 20 lines where the union is 60; and because both errors scale with how finely duplication is grouped, it rewarded an engine for fragmenting one clone class into several and penalised one for grouping correctly — which is the opposite of what the measure exists to encourage. Each occurrence is now

measured over the lines it covers rather than over the lines its class covers, since two token-identical copies separated by different numbers of comment or blank lines do not span equally many source lines.

Table 2: Methods, ordered by return on investment. R1–R5 are implemented and tested in phpcpd-next; R6–R8 are analyzed and declined, for the reasons given.

ID	Improvement	Capability added	Effort	Status
R1	SourcererCC token-bag + inverted index + IDF	Reorder-tolerant Type-3; near-linear scale; corpus-adaptive precision	Med	Done
R2	Type-aware edit weights in <code>filledBuffer</code>	Precision at high <code>--edit-distance</code>	Low	Done
R3	Toma token-type abstraction	Type-2 for both engines; feeds R2	Low	Done
R4	Inconsistent-clone reporting	Bug-finding output from discarded data	Low	Done
R5	Incremental / index-based detection	CI scalability $O(\text{diff})$	High	Done
R6	Multi-similarity + thresholds (+ opt. ML)	Recall tuning	Med	Declined (breaks determinism)
R7	Deckard AST vectors + LSH [6]	Syntactic precision	High	Declined (low gain)
R8	Neural / LLM semantic (Type-4)	New paradigm	High	Declined (rare class)

6 The Unified Engine

The third future-work direction of the first revision was the residual cost of suffix-tree matching: candidate enumeration, the part the banded dynamic program does not reduce. This section reports what replaced it. Rather than optimize enumeration inside one engine, we built a single engine whose enumeration is a *sample* with a proof attached, and folded the three methods above into it as measured baselines.

6.1 Design

The engine runs four stages over two views of each file — the raw token signature and a normalized one in which identifiers and literals fold to type classes.

Encode. A file becomes five bytes per significant token: a type byte and a hash of the token’s text. Nothing else survives, which is what makes the later stages cheap and what makes a per-file cache exact.

Seed by winnowing. Fingerprints are selected by winnowing [7] over $K = 16$ -token grams in a window W . The property that matters is not compression but the guarantee: any common run of at least $W + K - 1$ tokens contains a *selected* fingerprint in both copies, so sampling cannot lose a clone the tool is contracted to find. With the shipped window this threshold is $\lceil \text{min-tokens}/2 \rceil$.

Chain. Shared fingerprints become anchors; anchors are extended to maximal exact runs and chained by a sparse dynamic program [8] into colinear readings, one per file pair.

Classify and verify. A reading is exact, gapped, or reordered. Gapped readings are verified against an edit budget proportional to the span; the chain itself often witnesses a bound tight enough to decide the question without running the dynamic program at all, which removes the quadratic term from the common case.

Three additions distinguish the engine from a seed-and-extend detector.

A refused reading does not consume its evidence. A reading refused for dissimilarity says nothing about its parts — it is a verdict on one alignment, and the chain beneath it is made of exact runs. Refusing it and discarding its anchors therefore loses the exact clones inside it. The refused reading is instead decomposed at its widest interior gap, which is the largest single contributor to the cost that refused it, and the halves are re-asked. On one project’s own test suite this recovered every location a Rabin–Karp baseline reported that the engine had been missing.

An order-free channel for what seeding cannot reach. A seeded method needs a run long enough to anchor. A displacement shorter than K leaves no such run on either side of it, so a function whose statements have been permuted in place is invisible to seeding at any threshold — not by tuning but by construction. The engine therefore carries a second channel for that class and no other: per-function bags of three-token shingles [9], indexed under a prefix filter [10, 11] ordered by ascending document frequency, promoting bag-overlapping function pairs into the same classification path as anchor-seeded ones. A pair is accepted when the two bags match bijectively at coverage ≥ 0.7 [3] *and* the blocks that moved can be named; a finding that cannot say what moved is not reported. The shingle length is derived from the statement-length distribution rather than chosen: a statement shorter than the shingle contributes nothing that survives being moved, and over 8,680 statements in the four benchmark corpora, 90 percent are three tokens or longer.

Crucially, “displaced” is a change of *order*, not of offset. An inserted statement shifts every later run to a new offset while preserving every run’s order; only a permutation changes the order. The displaced set is therefore the complement of the heaviest order-preserving subsequence of matched runs.

Two precision tiers, both structural. Duplicate-looking findings on real applications concentrate in two shapes, and each has its own tier.

The first is the file: analyzing unwired code measures a project’s housekeeping rather than its duplication. A triage stage runs before any view is fingerprinted, and it separates what it can *prove* from what it can only estimate. Four rungs prove — a file is a derived artifact, is unreferenced, is shadowed by another declaration of the same symbols, or belongs to no manifest — and those discard. A naive-Bayes classifier over content and wiring features estimates, and therefore only *tags*: files it scores suspicious are marked with their score and their strongest features, counted, and still scanned. A wrong estimate then costs a tag rather than a finding.

The second is the span, and it is where three earlier attempts failed. Anchor multiplicity, tokens per line, and the fraction of a span that is not a literal were each measured and each refuted by the same counter-example: a generated parser’s action table is as literal as a configuration file and is a genuine clone of its sibling parser’s, so no cut on *how literal a span is* can remove data-table noise without removing it too. What separates them is not literalness but identity. Two different tables in two files are a real finding; a table matching *itself* has no second copy anyone could delete — the repetition is what makes it a table. The rule is therefore a membership test with no threshold: are these two spans two runs of elements of one and the same literal array, where “literal” means containing no computation at any depth?

6.2 What it costs and what it buys

On the permutation families of the benchmark, recall rises from 73.6 and 61.9 percent (adjacent and distant displacement) to 93.8 and 89.7, against the token bag’s 77.5 and 81.4 — the engine now leads the method it was built to absorb, on that method’s own strength. The guaranteed region is recalled entirely, at both sample sizes the harness draws. Determinism, incremental equivalence, and the chaining oracle are unchanged.

Wall-clock is the cost, and it is stated beside what it bought rather than on its own. Against the merged default pipeline the engine runs at 1.37 to 1.86 times the elapsed time at the largest sizes measured, against a target of 1.5 — the level is missed. What holds is the shape: the ratio *improves* monotonically with corpus size on every large application measured, 6.1 to 1.7 on one and 6.5 to 1.4 on another, and a ratio that degrades with size was always the property that would disqualify the method, since it is the one that makes a tool fail on the codebases that most need it. The cost is not slower findings but more of them: the same engine reports 725 clones on one corpus where it reported 617, and every additional accepted candidate is verification work the earlier build never did because its chain scorer had already truncated the pair. Self-similar input, not scale, is what the cost actually tracks, and it is the rule rather than an exception. Measured across all six corpora in one sitting, the engine is at $0.84\times$ the default pipeline on WordPress at 1,848 files — faster than the two engines it replaces — and at $11.9\times$ on PHPUnit, which is 45 percent larger. The two corpora it loses on are the two that carry large near-duplicate regions: PHPUnit’s cost concentrates in one directory of some ninety near-identical test methods, and 82 percent of `php-parser`’s runtime is two generated parser files, 18 percent of its lines, that are near copies of each other. Removing those two files takes the corpus from 6.4 to 1.1 seconds. The shape that disqualifies a method would be a ratio that degrades with the size of ordinary code; what these measurements show instead is a cost that concentrates in regions a reader would not refactor by hand either.

6.3 Precision, and a correction

An earlier measurement of this engine’s precision on a private application corpus reported 0.34 against the other engines’ 1.00 on the same rated sample. That number is superseded, and the reason is worth stating rather than quietly replacing.

The pool it was drawn from was taken from a live working tree. Between the rating pass and the close, that tree drifted — the pooled finding count moved from 74 to 1,030 — and roughly 30 percent of the rated pool turned out to lie inside a backup subtree that no longer exists. The two numbers were therefore measurements of two different corpora, and neither is reproducible from the other. Nothing about the rating was wrong; the corpus underneath it was not the same corpus twice.

Two changes follow, and both are methodological rather than cosmetic. Any figure destined for publication is now taken from a *pinned* snapshot: a recorded content-hash list, kept outside every repository, against which a tree can be re-verified before and after a measurement. And a comparison states one corpus and one configuration; where two figures come from different corpus definitions, both are reported as separate lines rather than subtracted. Applying the second rule retired a second number as well — an earlier report that the engine ran roughly 42 times slower than the baseline on one corpus turned out to have been measured on a file list a third of which was a trash directory the tool’s own defaults exclude.

The second pass has since been made, and the figure exists. On a fresh pool drawn from a pinned snapshot and rated blind by two raters ($\kappa = 0.811$, 55 of 60 agreements), the engine’s *asserted* stratum — the findings the tool claims are duplicated logic, as against those it reports with a structural label attached — stands at 0.639 [0.476, 0.775] under *both* raters, with its demoted stratum at 0.400 and 0.500. The stratification separates in the direction it was pre-registered to, so the label carries information rather than merely renaming.

The criterion had been fixed before the pool was drawn: both raters’ asserted-stratum precision at 0.80 or above. It is not met, and 0.80 lies above both upper interval bounds. The residual was then decomposed rather than left as a gap: of one rater’s thirteen asserted false positives, three are route-registration lists and two are class-constant data tables that the structural labels do not reach; counterfactually demoting both families reaches only 0.697 and then 0.742. The remaining eight sit on the boundary between an adapted copy and boilerplate that two careful raters have split on in every round of this protocol. The default therefore stays with the merged Rabin–Karp/token-bag pipeline, and — this being the point of pre-registration — the bar was not moved to accommodate the result.

A second study, on public corpora (2026-09-15). The round above was run on a private application corpus, which is the right corpus for the question it asks and the wrong one for anybody wanting to reproduce it. A later round was drawn from two of this paper’s public corpora, `php-parser` and `symfony/string`, 60 findings sampled from each, pooled across six engine variants with the engine and the stratum hidden from the raters. Two raters, and both rounds clear the agreement bar this protocol requires: $\kappa = 0.712$ on `php-parser` and 0.797 on `symfony/string`.

On the asserted stratum, Rabin–Karp with the type anchor scores 0.889 and 0.778 on `php-parser` and 0.923 and 0.846 on `symfony/string` (the two figures being the two raters); the unified engine scores 0.774 and 0.677, then 0.706 and 0.706. The ordering matches the private-corpus round — the contiguous matcher ahead of the unified engine on precision — and the conclusion is unchanged: the default stays with the merged pipeline.

The round also settled a question this paper had left open. The token bag under normalization scores 0.200 and 0.133 on `symfony/string`, twelve or thirteen of fifteen findings false, against 0.857 from both raters for the same engine on the raw view. The cause is structural rather than a threshold: a bag is order-free, and normalization folds every identifier and literal to a placeholder, so two data tables of the same shape become the same multiset. Order is what protects the contiguous matcher from the same collapse. That configuration has been removed from the tool, which is why Table 5 no longer recommends it and Figure 1’s `tb+fuzzy` series has no counterpart in the shipped engine.

6.4 A note on the learned component’s life cycle

The triage classifier is worth recording as a *method*, because what happened to it is likely to happen again to learned components in tools of this kind.

It was trained to answer a question the tool could not answer from first principles: which files are not

program text? It answered it well enough to be useful, and the log-odds table it produced was readable — which features carried the decision, and how much. That readability is what made the next step possible. Each feature the model relied on most was examined, and several turned out to be *provable* rather than merely predictive: a file with no referent under the project’s own manifest is not wired, and a file whose declared symbols are all declared by another loadable file is shadowed. Those became rungs that discard on evidence a reader can check, and they moved upstream of the classifier.

The consequence is that the classifier’s positive class was removed from underneath it: on the corpus it was trained for, every file it was built to catch is now caught by proof before it is consulted. It was retained for one release as a tag rather than a filter, under a condition fixed in advance — that its fate be decided at the next rating round on that round’s evidence — and that round decided it. Of the seven pooled findings its tag demoted, five were already demoted by a structural label that proves its case; the two it reached alone were rated genuine duplication by both raters. Its correct output was redundant and its exclusive output was error, so it was deleted: model, trainer, margin and tag. The triage stage is now proof on every rung.

The general shape — train a guesser, read what it learned, convert the parts that can be proven into rules, and let the guesser shrink toward the residue — is a reasonable default for a deterministic tool that must justify every finding it reports. A model that cannot be inspected could not have been mined this way.

7 Practical Recommendation: Which Mode by Default

We ran every variant against the original `phpcpd.phar` (v6.0.3, Rabin–Karp only) on six corpora at pinned SHAs, using the phar’s own defaults (min-tokens 70, min-lines 5). All numbers are percentage of total lines flagged as duplicated, corrected per the accounting formula above. Figure 1 and Table 4 report that result, and it is now a historical one: both halves of the comparison have since stopped being commensurable, for two independent reasons.

The first is the token model. This engine subsequently stopped discarding single-character tokens — `;`, `(`, `=` and every operator, about half the program text and the half that says what the code does — and raised its own `--min-tokens` default from 70 to 100 in consequence. A “70-token” clone therefore means one thing to the phar and another here, and the comparison’s shared threshold no longer selects comparable populations. The second is the accounting: this engine now charges only lines holding a token the matchers can see, so a docblock inside a clone’s line range is no longer counted as duplicated, while the phar counts the whole range. The duplicated-line percentages are consequently not two measurements of one quantity.

Neither can be repaired by re-running at a common setting, because there is no common setting: the two tools disagree about what a token is and about what a duplicated line is. What can honestly be compared is the two tools *as each is configured out of the box*, which is also how a user meets them. Table 3 reports that, measured 2026-09-15.

Table 3: The two tools as each ships: `phpcpd.phar` v6.0.3 at its defaults (min-tokens 70, min-lines 5) against `phpcpd-next` at its own (min-tokens 100, min-lines 5, type-anchored normalization, `rk+tb` merged). Clone counts are counts of reported findings. The percentages are each tool’s own duplicated-line figure and are *not* comparable across the two halves — the two count duplicated lines differently, which is the point made above — and are printed so each half can be read against its own tool’s output. Typeset from `bench/results/out-of-the-box.tsv`, which `bench/run-out-of-the-box.php` writes.

corpus	phar clones	phar %	next clones	next %
firefly-iii	95	1.54	503	12.52
php-parser	15	4.15	44	4.07
phpunit	156	4.60	661	15.84
symfony-console	7	0.27	137	5.59
symfony-string	1	0.33	47	22.96
wordpress	177	1.51	1,010	5.40

The gap is large and is mostly the token model rather than the engines: seeing operators makes a fixed token budget reach about half as far into the source, so a threshold of 100 new tokens admits shorter and more numerous fragments than 70 old ones. It is not evidence that either tool is finding more real

duplication than the other, and we do not read it that way. What the table does establish is that the two are no longer substitutable at any threshold, and that the equivalence this section originally claimed — that **rk** finds exactly the clones the phar finds — has to be withdrawn rather than restated at a different number.

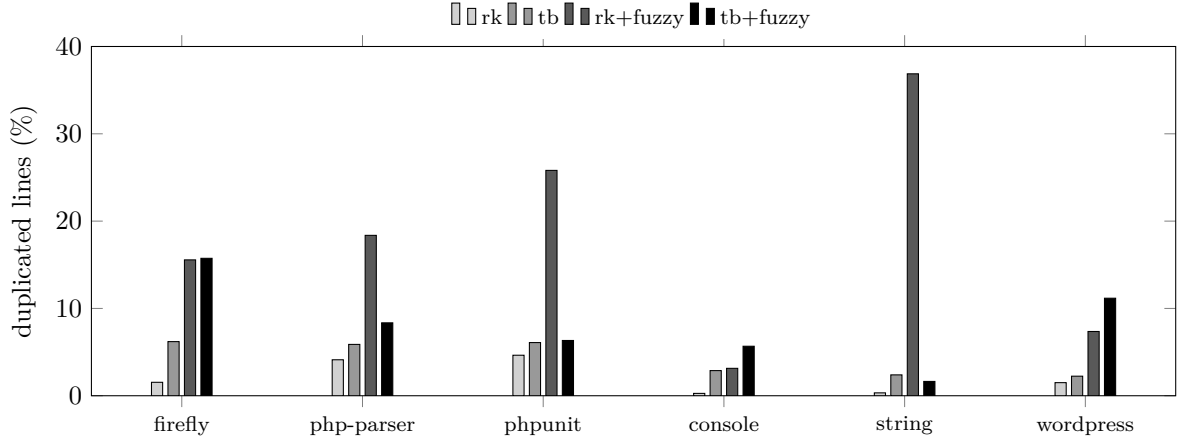


Figure 1: Duplicated-line percentage by algorithm variant across six corpora, *as measured when recorded*. **rk** found exactly the clones the original phar finds on all six; that equivalence no longer holds, and **tb+fuzzy** is no longer a configuration the tool has — see the errata after the abstract. **tb** adds a controlled increment (reordered blocks); **--fuzzy** variants inflate the reported rate (up to 37% on **symfony/string**), which is why fuzzing is a hidden research flag, not a default.

Table 4: Duplicated-line % per corpus and variant, as measured when recorded. **rk** reported exactly the clones the original phar reports on all six corpora; the totals differ where the two count duplicated lines differently (§4). The equivalence no longer holds — see the errata after the abstract.

corpus	phar	rk	rk+fuzzy	tb	tb+fuzzy
firefly-iii	1.54	1.54	15.56	6.20	15.74
php-parser	4.15	4.12	18.37	5.88	8.35
phpunit	4.60	4.64	25.81	6.09	6.33
symfony-console	0.27	0.27	3.14	2.88	5.67
symfony-string	0.33	0.33	36.87	2.39	1.64
wordpress	1.51	1.50	7.36	2.24	11.17

Findings. (1) **rk** reports exactly the clones the original phar reports — the same count on all six corpora, at the same locations and lengths. The duplicated-line totals differ by 0 to 0.8 percent, and differ by construction rather than by disagreement: the phar sums a per-clone contribution where this work measures clone coverage, so the two answer different questions about the same clones. (2) **rk+fuzzy** is unsuitable as a default: 3–80× the baseline with near-zero precision gain. (3) **tb** is the best complement to **rk**: it adds the reordered clones **rk** misses while staying within 1.5–4× the baseline, kept precise by the IDF filter. (4) **tb+fuzzy** is corpus-dependent and not a safe default. (5) the suffix tree, even after the banded-DP optimization below cut its matching cost $\sim 3.5\times$, stays above interactive speed on corpora over a few hundred files, so it is retained for targeted studies, not the default. Read together, these place **rk+fuzzy** as a dominated configuration — its volume increase is inferred false positives, consistent with the dominance result of §8 — and **rk+tb** as the non-dominated choice on the precision–cost frontier; because the six public corpora carry no labelled recall, we claim consistency with that frontier rather than optimality on it.

Optimizing the suffix tree. Rather than adopt plausible but untargeted advice (object-free node storage and Ukkonen construction, both already present), we profiled. Construction is $O(n)$ and negligible; the approximate-matching edit-distance DP accounts for roughly 70% of **findClones** and grows super-linearly with **maxErrors**. By the band-sufficiency lemma (§2), the full $O(L^2)$ matrix is unnecessary:

restricting computation to the diagonal band of width $2\text{maxErrors} + 1$ (the Ukkonen cutoff) lowers it to $O(L \cdot \text{maxErrors})$ with identical output. On a Firefly III slice this reduced `findClones` by $\sim 3.5\times$ at every edit distance with byte-identical clone output; the residual cost is candidate enumeration, which banding does not address. The change was admitted only after the benchmark confirmed unchanged results — the same evidence-before-adoption rule applied throughout.

Default mode. `phpcpd <dir>` now runs **rk + tb combined** (exact + reorder-tolerant, merged with deduplication); `--rk` restricts to Rabin–Karp only. Research flags (`--fuzzy`, `--algorithm`, ...) remain parseable but hidden from `--help` to prevent accidental production use (Table 5). Since the 2.0.0 release the default also runs corpus triage in a posture that *discards* — it removes from the scan those files the project’s own manifest and symbol table show to be unwired, shadowed, vendored or generated, so duplication is measured over what the project actually runs. `--trriage-posture=label` keeps the earlier behaviour, stamping the same files and changing no finding, no file and no exit code.

Table 5: Recommended invocation by goal.

Goal	Command	Output
Daily DRY check	<code>phpcpd src/</code>	rk+tb, clean signal
Exact clones (CI speed)	<code>phpcpd --rk src/</code>	matches original phar
Type-3 gapped study	<code>phpcpd --algorithm unified src/</code>	divergent ranges named
Type-2 research	<code>phpcpd --algorithm tokenbag src/</code>	reordered blocks only

8 Evaluation: BCB-PHP

8.1 BCB-PHP: a labelled PHP clone benchmark

A study of clone-detection methods needs labelled ground truth, and for PHP none existed: the standard benchmarks are language-bound — BigCloneBench [12] is Java, POJ-104 is C — leaving PHP with no equivalent. Without one, the methods of §2–5 could be implemented but not honestly compared. We therefore built **BCB-PHP**: a mutation-injection harness that generates ground-truth Type-1/2/3 variants from real PHP across a curated, pinned corpus, following SourcererCC’s injection-based ground-truth approach [3]. To respect corpus licences (WordPress GPLv2, Firefly III AGPLv3, the rest permissive) the released artifact is a fetch-script + commit SHAs + injection manifests + harness — never redistributed source.

Building the benchmark is a contribution in its own right, and we release it for reuse. Its corpora span a deliberate type-density gradient (§8.2), from traditional untyped PHP (WordPress) to PHPStan/Larastan-max typed codebases (PHPUnit, Firefly III). That axis lets others study questions this paper only begins to answer: how clone detection behaves on *modern typed* versus *traditional untyped* PHP, and how detection methods differ from one another on the same labelled inputs.

8.2 Type-density is the independent variable

The thesis “type information improves clone detection” is proved not by one typed corpus but by showing detection quality as a function of *how typed* the code is. Corpus type-density is thus the experiment’s *x*-axis and untyped code is the control end of it.

Definition 1 (signature type-density). $\text{td} = (\text{typed signature slots})/(\text{total signature slots})$, where slots are function/method parameters and return-type positions. The signature surface is unambiguously tokenizable; property declarations are excluded to avoid guessing class context.

We compute the map rather than assert it (Figure 2, Table 6). Clone-density is measured by the Rabin–Karp engine, independent of the type axis. The predictions hold: WordPress anchors the untyped end (13.3%), the PHPStan/Larastan-max codebases (PHPUnit 92.7%, Firefly III 90.7%) the typed end, with a clean spread between — a usable axis, not a hunch.

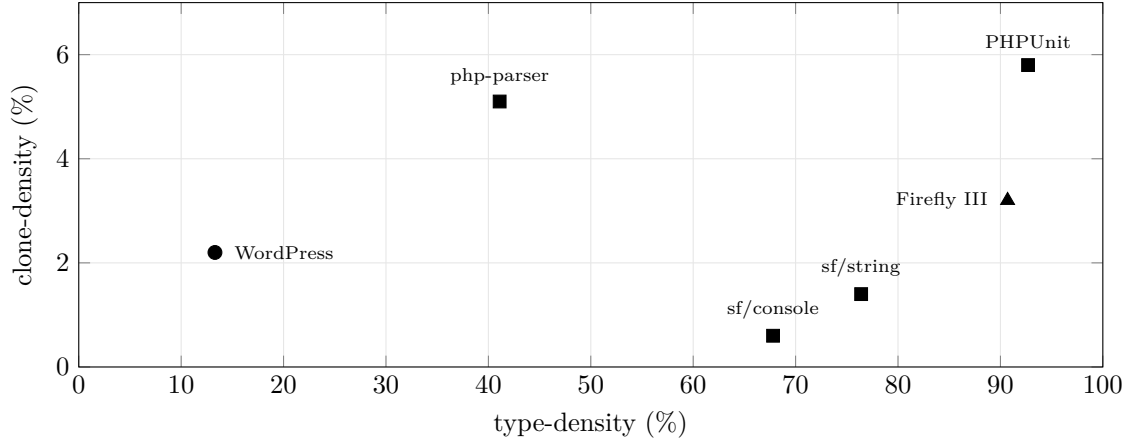


Figure 2: Type-density $\times$ clone-density map (full corpora, pinned SHAs). Marks: $\bullet$ untyped control, $\blacksquare$ injection substrate, $\blacktriangle$ ecological corpus. The scarce typed-and-cloney quadrant is occupied by an application (Firefly III), not a library.

Table 6: Computed type-density / clone-density map (bench/measure-density.php).

corpus	files	type-dens.	clone-dens.	bucket
WordPress	1845	13.4%	2.2%	untyped control
nikic/php-parser	342	41.1%	5.1%	injection (dependency)
symfony/console	359	67.8%	0.6%	injection
symfony/string	33	76.4%	1.4%	injection
Firefly III	1445	90.7%	3.2%	ecological
PHPUnit	2689	92.6%	5.7%	injection (reflexive)

8.3 E2 — the effect of type information

The decisive operator, possible only in typed PHP, clones a block and changes an `int`-typed variable to `string`-typed with structure held identical. Name-blind normalization calls these clones (false positive); type-anchored normalization (`--type-anchored`, which preserves built-in type keywords instead of folding them to ID) correctly separates them. Injecting at *function* granularity over the full eligible population (functions carrying an `int/float` hint, ≥ 25 tokens), Figure 3 and Table 7 report specificity.

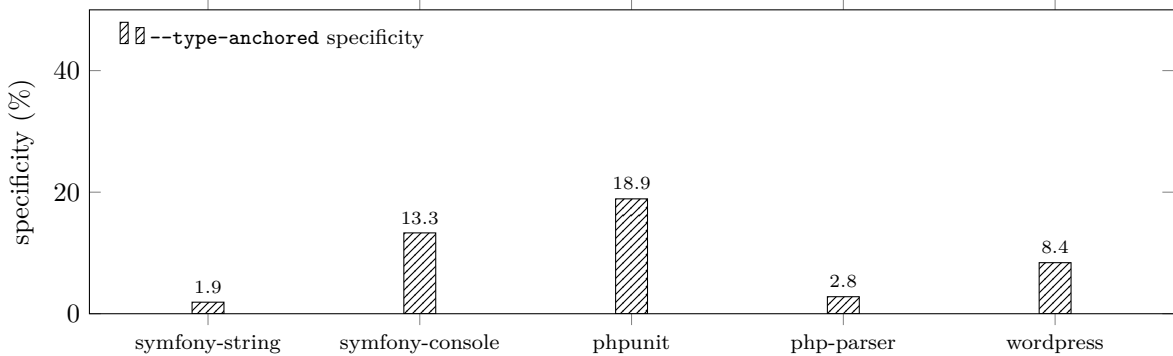


Figure 3: Specificity on same-shape-different-type pairs. Name-blind `--fuzzy` is a flat 0% floor (it calls every such pair a clone, so it is omitted from the bars); `--type-anchored` lifts specificity by +1.9 to +18.9 points at *zero* recall cost (recall = 100% in both modes, Table 7).

The lift is a property of *type-load-bearing* functions, not of corpora: PHPUnit shows the largest lift and php-parser one of the smallest, which inverts what an earlier measurement of this experiment reported and is worth stating plainly. That measurement predates the change described in §7 that made single-character tokens visible to the matcher — roughly half the program text. A typed signature is a fixed number of tokens, so widening the stream it sits in makes it a smaller share of every function,

Table 7: E2: type-anchored normalization over the full eligible-function population. Recall is identical in both modes; the lift is pure specificity gain.

corpus	fns	spec. fuzzy	spec. anchored	lift	recall
symfony-string	52	0.0	1.9	1.9	100
symfony-console	165	0.0	13.3	13.3	100
phpunit	238	0.0	18.9	18.9	100
php-parser	179	0.0	2.8	2.8	100
wordpress	107	0.0	8.4	8.4	100

and type-anchoring has proportionally less to bite on: the lift is real, dominance still holds, and it is smaller than it looked when operators were invisible. `php-parser` feels that most because its node and visitor methods are short, so their signatures carried the most weight before and lose the most now. On the census the corpus-level benefit no longer tracks type-density as it did (Pearson $r = 0.43$, Spearman $\rho = 0.30$ over the five corpora): `symfony/string` is the second most densely typed and shows the smallest lift, while WordPress is the least densely typed and shows more than `php-parser`. The scatter of Figure 2 is reported as the weak association it is rather than as a gradient, and the per-corpus story still needs *two* figures, because a lift concentrated in short typed functions is not a property a single corpus-level number carries. Because the two modes share the same recall, this is a clean dominance relation: on same-shape-different-type pairs, type-anchored normalization *Pareto-dominates* name-blind fuzzing — equal recall at strictly higher specificity — the strongest form the claim can take, since here both axes are measured against injected ground truth.

8.4 E3 — inconsistent clones in practice

`bench/run-e3.php` scans the ecological corpus (Firefly III, 200 most-recently-modified files) once through the suffix tree (`--algorithm suffixtree --edit-distance 3 --min-tokens 50`), reads the gapped clones via `CodeClone::isGapped()` (the R4 capability), deduplicates by file pair, and cross-references git: the *staleness gap* is the number of days between the two copies’ last commits — a high gap means one copy was maintained long after its sibling, the canonical inconsistent-clone risk. The scan found 68 inconsistent clones and two distinct diverged-copy pairs (Table 8); both were manually verified as real. Exact-match output (Rabin–Karp / `--rk`) flags neither as inconsistent, having no gapped-clone concept.

Table 8: E3 diverged-copy pairs in Firefly III (200-file slice), ranked by staleness. Both verified manually against source.

#	copy A	copy B	lines	stale (d)	verdict
1	Navigation-AddPeriodTest	Navigation-PreferredSql-FormatTest	4	35	Real — duplicated <code>setUp()</code> scaffold across two tests of the same class; a shared base would remove it.
2	Autocomplete/-CurrencyController	Autocomplete/-TagController	25	9	Real — identical DI constructor + search loop; <code>Currency</code> later gained a deprecated endpoint <code>Tag</code> never received.

9 Availability

`phpcpd-next` is openly available under the permissive MIT licence. It began as a fork of `sebastianbergmann/phpcpd`, archived in 2023 and then BSD 3-Clause; every inherited file has since been rewritten from its specification, each proved to emit byte-identical output before its attribution was changed, and `bench/check-provenance.php` measures that inventory and reports it at zero. It has zero runtime dependencies (`php >=8.5` plus `ext-dom/ext-mbstring`) and projects one format-neutral model (`CodeCloneMap`) to Text, PMD-CPD XML, JSON, and SARIF 2.1.0 — the last ingested natively by GitHub Code Scanning, with gapped (inconsistent) clones emitted at **warning** severity and exact clones at **note**, so the bug-bearing clones surface as higher-severity PR annotations. BCB-PHP ships as a

fetch-script + pinned SHAs + injection manifests + harness; every result is reproducible from the SHAs and `composer.lock` files, and the modernization is recorded diff-by-diff in `docs/MODERNIZATION.md`.

10 Conclusion

Starting from `phpcpd`’s Rabin–Karp exact matcher and its unreleased suffix-tree engine, we asked an empirical question: of the clone-detection methods available, which are appropriate for a fast PHP tool? Answering it required infrastructure that did not exist — a labelled PHP clone benchmark — so we constructed one (BCB-PHP) while modernizing and completing the inherited engines and implementing additional literature-grounded methods. The results — a type-density gradient, a +1.9 to +18.9 point specificity increase from type-aware normalization at no recall cost, a six-corpus method comparison, and confirmed inconsistent clones in an application corpus — support a default that combines exact and reorder-tolerant matching (`rk+tb`), retains fuzzing only for research, and reserves approximate suffix-tree matching for targeted analysis. Those three engines became the *measured baselines* for a fourth (§6) that subsumes what each was kept for: exact matching, reorder tolerance, and gapped matching with the divergent ranges named rather than a boolean. Which of them a release ships as the default was put to a criterion fixed before the measurement, and answered: the criterion is not met (§6.3), the merged pipeline remains the default, and the suffix-tree engine — the one component of the package that was not its authors’ own work — was removed in 2.0.0 once the fourth engine covered what it was kept for.

Future work. The component most likely to outlast any single result is the benchmark, since it turns open questions about PHP clone detection from matters of argument into measurable ones. Three directions follow from the evidence. First, the injection operators can be broadened beyond the `int/float` confusion studied here to a wider class of type-level edits (`bool↔int`, nullable versus non-nullable, and class or interface substitutions), enlarging the type-load-bearing population and testing whether the specificity gain of type-anchored normalization generalizes across type kinds. Second, the harness can be applied to further real-world and proprietary codebases to assess external validity beyond the six public corpora. Third, the residual cost of suffix-tree matching — candidate enumeration, the part the banded DP (§7) does not reduce — was the engine’s main scaling limit on large corpora, and §6 reports what replaced it: enumeration became a sample with a proof attached rather than an exhaustive pass to be optimized. What remains open there is not enumeration but verification cost on self-similar files, and the precision boundary of §6.3 — a residue that three rating rounds have located in the raters’ own disagreement rather than in any mechanical rule, which suggests the next instrument should measure a *ranked* report rather than a flat pool. In each case the methodological commitment is the one adopted throughout: a candidate improvement is admitted on PHP-specific empirical evidence, not on its theoretical appeal.

References

- [1] C. K. Roy and J. R. Cordy, “A survey on software clone detection research,” Queen’s Univ. Tech. Rep. 2007-541, 2007.
- [2] E. Juergens, F. Deissenboeck, B. Hummel, and S. Wagner, “Do code clones matter?” in *Proc. ICSE*, 2009, pp. 485–495.
- [3] H. Sajjani, V. Saini, J. Svajlenko, C. K. Roy, and C. V. Lopes, “SourcererCC: scaling code clone detection to big-code,” in *Proc. ICSE*, 2016, pp. 1157–1168. arXiv:1512.06448, 1603.01661.
- [4] Y. Wu *et al.*, “Toma: a token-type-aware clone detector” (token-type abstraction, six similarity measures, “simple clones dominate”), in *Proc. ICSE*, 2024.
- [5] B. Hummel, E. Juergens, L. Heinemann, and M. Conradt, “Index-based code clone detection: incremental, distributed, scalable,” in *Proc. ICSM*, 2010, pp. 1–9.
- [6] L. Jiang, G. Misherghi, Z. Su, and S. Glondu, “DECKARD: scalable and accurate tree-based detection of code clones,” in *Proc. ICSE*, 2007, pp. 96–105.
- [7] S. Schleimer, D. S. Wilkerson, and A. Aiken, “Winnowing: local algorithms for document fingerprinting,” in *Proc. SIGMOD*, 2003, pp. 76–85.

- [8] D. Eppstein, Z. Galil, R. Giancarlo, and G. F. Italiano, “Sparse dynamic programming I: linear cost functions,” *J. ACM*, vol. 39, no. 3, pp. 519–545, 1992.
- [9] A. Z. Broder, “On the resemblance and containment of documents,” in *Proc. Compression and Complexity of Sequences*, 1997, pp. 21–29.
- [10] S. Chaudhuri, V. Ganti, and R. Kaushik, “A primitive operator for similarity joins in data cleaning,” in *Proc. ICDE*, 2006, p. 5.
- [11] C. Xiao, W. Wang, X. Lin, J. X. Yu, and G. Wang, “Efficient similarity joins for near-duplicate detection,” *ACM Trans. Database Syst.*, vol. 36, no. 3, pp. 1–41, 2011.
- [12] J. Svajlenko, J. F. Islam, I. Keivanloo, C. K. Roy, and M. M. Mia, “Towards a big data curated benchmark of inter-project code clones,” in *Proc. ICSME*, 2014, pp. 476–480.